

WikiSkill: 将智能体经验编译为持久知识以实现技能进化

Liyan Tang¹, Cyrus Rashtchian¹, Chun-Sung Ferng¹, Andrew Tomkins¹, Da-Cheng Juan¹ and Tu Vu^{1,2}

¹Google Research, ²Virginia Tech

智能体技能将专业知识和流程打包为可复用资源，以扩展人工智能智能体的能力。近期工作从智能体经验中自动发现此类技能，使智能体能够通过交互逐步适应。然而，指导技能发展的洞见通常分散在优化历史中，限制了它们在迭代间的系统复用。本文提出 WikiSkill，一个使智能体技能与持久知识库（维基）共同进化的框架。在高层面上，WikiSkill 将原始执行经验、累积知识和可执行技能分离，同时持续将经验整合到维基中，后续技能更新可在此基础上构建。在多种基准和模型上，WikiSkill 一致地优于最先进的技能进化方法，并在大多数模型-基准设置中优于无技能基线。我们发现技能进化与模型扩展互补：更大的模型通常从进化技能中获益更多，而具备技能的小模型可以显著优于不具备技能的大得多的模型。我们还发现进化技能在不同模型和模型家族间有效迁移，且由其他模型进化的技能可以优于自我进化的技能。最后，我们的消融研究证实，维基中的持久知识积累对于有效的技能进化至关重要。这些结果证明了系统积累和精炼智能体经验以开发可复用和可迁移技能的益处。

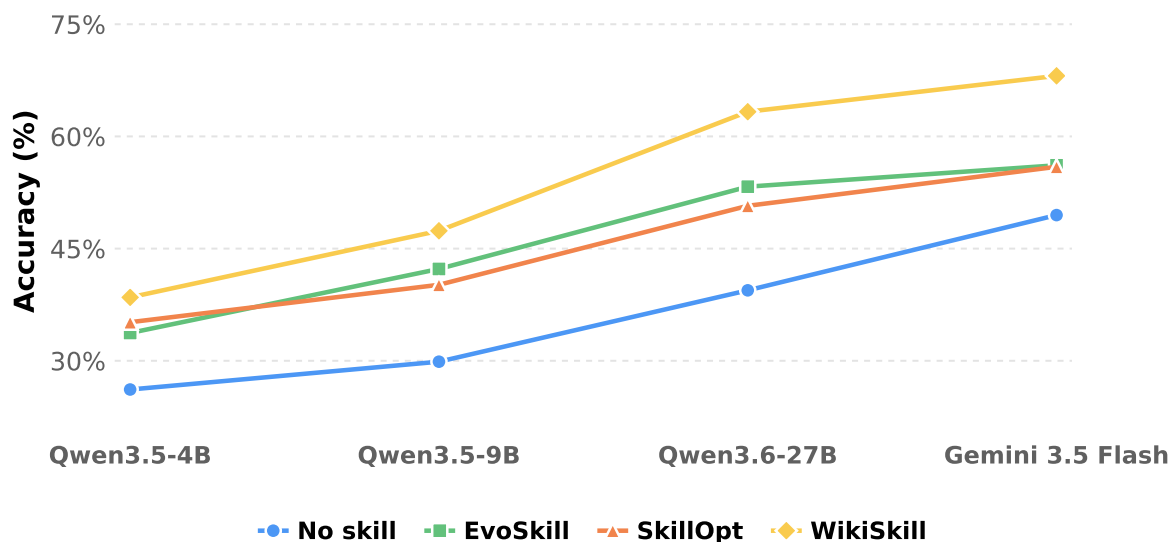


图 1 | WikiSkill 一致地优于无技能基线和现有技能进化方法。有趣的是，其优势在更强的模型上更为显著。我们报告了每个模型在评估基准上的平均准确率，使用无技能或由 EvoSkill、SkillOpt 和 WikiSkill 进化的技能（详见表 1）。

1. 引言

通用人工智能智能体（General-purpose AI agents）在跨领域执行复杂任务方面日益强大（Jackson 等，2025；Merrill 等，2026；Patwardhan 等，2026；Phan 等，2026）。然而，可靠地完成现实世界任务通常需要领域特定的专业知识（例如，程序性知识和工作流程）。智能体技能（Agent skills）（Chen 等，2026a；Li 等，2026；Liu 等，2026；Zhang 等，2025）提供了一种轻量级、开放的格式，用于在不更新模型参数的情况下捕获此类专业知识。其核心在于，技能将指令、脚本和其他资源打包成一个可复用的基于文件系统的模块（即一个有组织的目录）（Anthropic，2026；Li 等，2026；Xia 等，2026b；Zhang 等，2025）。这种设计使得专业知识具有一致性、可审计性，并且可以在兼容技能的智能体之间复用。它还支持渐进式披露（progressive disclosure）（Jiang 等，2026），即智能体在任何给定时间只加载相关内容，从而节省上下文空间。更广泛地说，技能提供了一种独立于模型参数积累知识的自然机制。

然而，开发有效的技能仍然具有挑战性。大多数智能体技能是人工编写的，这需要预测智能体将需要的程序性知识和工作流程（Li 等，2026；Liang 等，2026；Xu 和 Yan，2026）。这一挑战推动了近期的工作，这些工作通过在训练任务上执行智能体、分析成功和失败的轨迹，并根据所得经验改进技能，来迭代地开发智能体技能（Agrawal 等，2026；Alzubi 等，2026；Ni 等，2026；Ouyang 等，2026；Yang 等，2026；Yuksekgonul 等，2025）。

一个关键的设计问题是如何在技能演化过程中保存和组织智能体所学到的内容。先前的工作以不同方式解决了这一问题。EvoSkill（Alzubi 等人，2026）维护了先前提案及其评估结果的累积历史；Trace2Skill（Ni 等人，2026）从执行轨迹中提取并整合经验教训，用于技能更新；SkillOpt（Yang 等人，2026）则利用被拒绝的编辑反馈和逐轮次的元指导。然而，这些方法并未将已学到的内容作为独立且不断演化的知识表示加以维护。受 Karpathy（2026）关于大语言模型维基（LLM Wiki）观点的启发，该观点主张将经验汇编为持久且不断累积的知识，我们提出如下问题：智能体的经验能否同样被汇编为持久知识，以支持长期的技能演化？我们引入了 **WikiSkill**，它在原始经验和可执行程序（即技能）之间增加了一个结构化知识层。该层使得技能开发能够建立在跨迭代日益得到充分支持和整合的知识之上，而非建立在散落于技能演化工件中的知识之上。

WikiSkill 将智能体工作区组织为三个层次：原始层（Raw Layer）存储不可变的执行轨迹，维基层（Wiki Layer）维护结构化知识，技能层（Skill Layer）包含不断演化的程序性知识（图 2）。每次迭代包含四个组件：推理智能体（Inference Agent）使用当前技能执行推演，维基维护器（Wiki Maintainer）将轨迹整合到维基中，技能提议器（Skill Proposer）利用维基和轨迹提出技能更新，以及门控与回滚机制（Gating and Rollback）保留能提升验证性能的更新。虽然技能更新可以回滚，但维基会持久保留，以便未来的更新能够建立在累积的知识之上。从高层次来看，这些组件形成一个持续循环，在该循环中，经验被整合为持久知识，从而支持技能演化。

我们在五个基准上评估了 WikiSkill，涵盖数学推理（LiveMathematicanBench（He 等，2026））、网络搜索（SealQA（Pham 等，2026））、电子表格操作（SpreadSheetBench（Ma 等，2024））、长上下文文档问答（OfficeQA（Singhvi 等，2025））以及交互式具身任务（ALFWorld（Shridhar 等，2021）），使用了来自 Qwen（Qwen 团队，2026a，b）、Gemma（Gemma 团队，2026）和 Gemini（Google DeepMind，2026）系列的五个模型。我们发现 WikiSkill 优于现有的技能演化方法，并在大多数设置中比不使用技能有所改进。有趣的是，技能演化与模型扩展相辅相成。

在 Qwen 系列中，WikiSkill 将 4B、9B 和 27B 模型的平均性能分别提高了 12.3%、17.5% 和 23.9%，且增益随模型规模增大而增加。同时，演化后的技能可以弥补显著的模型规模差距：使用 WikiSkill 的 Qwen-3.5-9B 优于不使用技能的 Qwen-3.6-27B (47.4% 对 39.4%)。我们进一步发现，演化后的技能能够跨模型系列有效迁移，并且可以优于自演化技能。例如，在 ALFWorld 上，Qwen-3.5-9B 使用 Qwen-3.6-27B 演化的技能达到 70.2%，而使用自身技能为 63.4%。这些结果表明技能发现和技能执行是两种不同的能力。最后，我们的分析表明持久化 wiki 对这些增益至关重要，支持了我们的假设：在迭代过程中积累和精炼知识能够改进技能演化。

总之，我们的主要贡献如下：

- 我们提出了 WikiSkill，这是一个将智能体技能与持久知识库共同演化的框架，该知识库持续组织并精炼来自智能体经验的知识。
- 我们在五个基准和五个模型上证明，WikiSkill 始终优于现有的技能演化方法，消融实验证实了持久知识积累的重要性。
- 我们系统地研究了演化技能与模型能力之间的相互作用，表明技能演化与模型扩展相辅相成，并且演化后的技能能够跨模型有效迁移，有时优于自演化技能。

综上所述，我们希望我们的工作能够推动更多关于智能体如何从经验中积累、组织和重用知识的基础研究。

2. 问题设定

我们将大型语言模型智能体的迭代技能演化任务形式化。设 $\mathcal{D} = \{(x_i, v_i)\}_{i=1}^N$ 为一个任务数据集，其中 x_i 表示一个任务实例， v_i 表示其真实答案。我们将 $\mathcal{D}$ 划分为三个互不相交的划分：训练任务 $\mathcal{D}_{\text{train}}$ 、验证任务 $\mathcal{D}_{\text{val}}$ 和测试任务 $\mathcal{D}_{\text{test}}$ 。

智能体 π 是一个基于大型语言模型的系统，配备了一组工具 $\mathcal{U}$ （例如，命令行终端、搜索应用程序编程接口或文件读取器）和一个活跃技能集 $S = \{s_1, s_2, \dots, s_M\}$ 。技能是一个模块化的、基于文件系统的目录，将领域特定的程序性知识打包为指令、脚本和其他资源（Chen et al., 2026a; Li et al., 2026; Liu et al., 2026; Zhang et al., 2025）。具体而言，每个技能包含一个 SKILL.md 文件，其中包含前置元数据（唯一名称和简洁描述）以及完整的程序性指令和适用条件。技能集 S 初始化为空 ($\emptyset$)，并通过演化过程为每个数据集开发。

当执行任务 x_i 时，智能体接收任务上下文 x_i 并访问可用技能 S 。智能体在多个步骤中与环境交互，使用其工具和技能生成执行轨迹 $\tau_i \sim \pi(x_i; S)$ 。轨迹 $\tau_i = (o_1, a_1, o_2, a_2, \dots, o_T, a_T)$ 由观察 o_t 和动作 a_t 组成（可能包括对 $\mathcal{U}$ 中工具的调用）。最终动作 a_T 输出预测答案 $\hat{y}_i$ 。预测的正确性由领域特定的评分函数 $f(\hat{y}_i, v_i) \in [0, 1]$ 评估。对于任何任务划分 $\mathcal{D}_{\text{split}} \subset \mathcal{D}$ ，在 $\mathcal{D}_{\text{split}}$ 中的所有任务实例上展开智能体 $\pi(\cdot; S)$ ，得到相应的执行轨迹集合 $\mathcal{T}_{\text{split}} = \{\tau_i \sim \pi(x_i; S)\}_{(x_i, v_i) \in \mathcal{D}_{\text{split}}}$ 。任务划分 $\mathcal{R}(\mathcal{T}_{\text{split}})$ 上的性能是 $(x_i, v_i) \in \mathcal{D}_{\text{split}}$ 中所有任务实例的平均得分。

在 WikiSkill 中，迭代 k 时的系统状态由元组 (S_k, W_k) 表示，其中 $S_k = \{s_1, \dots, s_M\}$ 表示活跃程序性技能集， W_k 表示持久知识库 (Wiki)。虽然候选技能更新需经过验证门控并在得分下降时回滚，但知识库 W_k 在迭代间持续存在并累积。从 (S_0, W_0) 开始

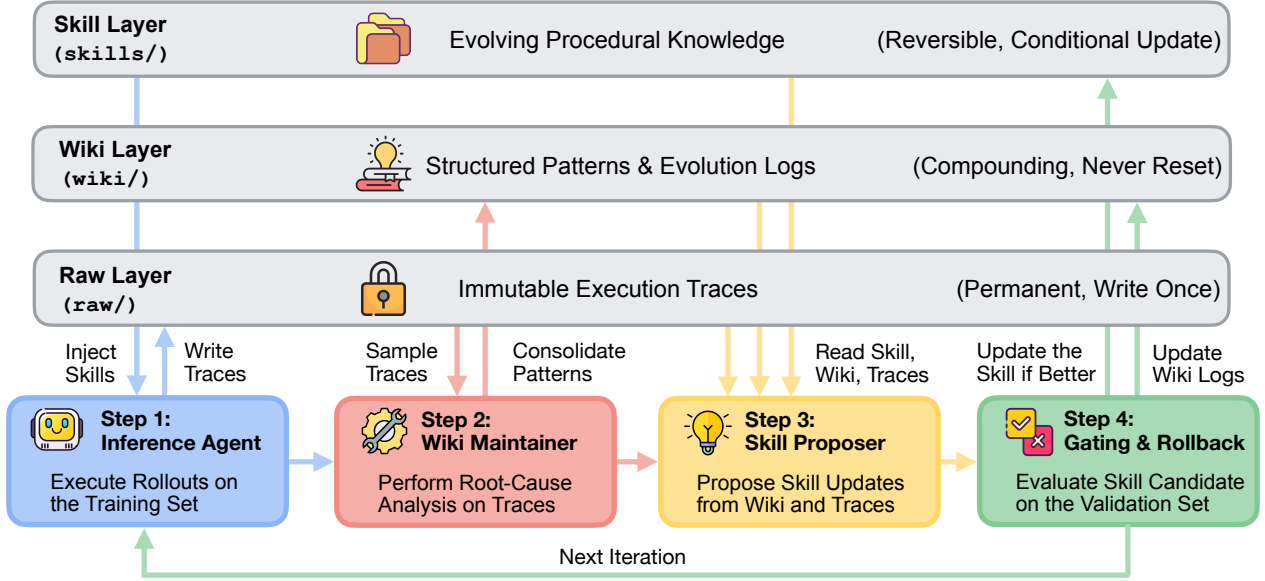


图 2 | WikiSkill 框架概览。智能体工作区分为三层：不可变的执行迹（原始层）、跨迭代累积的持久知识库（维基层）以及活跃的程序性指令（技能层）。在每个进化循环中，推理智能体执行展开（注入活跃技能但限制维基访问），维基维护者将迹整合进维基，技能提议者（采用 ReAct 机制）提出更新建议，而维基在所有迭代中得以保留。

$(\emptyset, \emptyset)$, WikiSkill 在迭代 $k \in \{1, \dots, K\}$ 中协同进化联合状态 (S_k, W_k) ，利用训练展开 $\mathcal{T}_{\text{train},k}$ 、模式整合以及基于 $\mathcal{T}_{\text{val},k}$ 的验证门控，以最大化在未见任务 $\mathcal{D}_{\text{test}}$ 上的最终测试性能 $R(\text{Ttest})$ 。

3. 方法论

本文提出 WikiSkill，一个协同进化智能体技能与持久知识库（维基）的框架。WikiSkill 围绕三层知识架构（§ 3.1）构建，执行一个精心编排的进化循环：智能体运行展开，维基维护者整合迹并更新维基，技能提议者提出技能更新，门控机制过滤变更（§ 3.2）。

3.1. 三层知识架构

WikiSkill 工作区由三个不同的层组成，如图 2 所示，并在下文描述。

原始层 (raw/) 该层存储每次迭代中从训练示例收集的原始执行迹 $\tau_i \in \mathcal{T}_{\text{train},k}$ 。这些迹捕获智能体完整的逐步交互，包括推理、工具调用、工具调用输出和最终答案。在我们的设置中，维基维护者和技能提议者智能体可以访问这些原始迹以分析智能体行为。为保留原始历史，该层不可变。

维基层 (wiki/) 该层将原始轨迹编译为结构化的、不断累积的知识，并在技能演化过程中持续维护。它包含一个模式目录 (patterns/)，其中填充了

以独立的标记文件记录具体的失败模式或成功策略，并附上可操作的变通方案。关键的是，该层通过演化日志（logs.md，由维基维护者更新）和技能影响追踪器（skill-impact.md，由外环框架在验证门控后程序化更新），在优化迭代中提供长期的历史感知。这些记录使维基维护者和技能提议者能够（1）观察完整的技能接受历史，从而避免再次提出已被拒绝的干预措施，（2）追踪先前迭代中提出的内容及其成功与否，以及（3）识别哪些错误在迭代间反复出现。维基在迭代之间不会被重置，而是在整个演化过程中持续积累和汇编知识。

技能层（技能层（Skill Layer）该层包含进化技能的活跃集合 S ，其中编码了推理智能体（Inference Agent）可读取的程序性知识。WikiSkill 中的每个技能目录包含两个文件：SKILL.md，包含技能的完整内容；以及 PURPOSE.md，将技能映射回激发其创建或修改的维基模式。技能层与维基层之间交互的详细信息见图 3，并在第 5.3 节的案例研究中加以说明。

3.2. 进化智能体与维基编排

WikiSkill 循环由四个组件构成。在每次迭代中，推理智能体（Inference Agent）（§ 3.2.1）使用 skills/ 中的活跃技能执行任务，在 raw/ 中生成不可变的执行轨迹。在训练回放期间，推理智能体被限制访问 Wiki 层，因为我们的消融实验（§ 5.1）表明，训练期间允许访问 Wiki 会对技能发展产生负面影响。接下来，Wiki 维护者（Wiki Maintainer）（§ 3.2.2）分析这些原始轨迹以及现有的 wiki/ 层，以诊断失败并提取成功策略，更新持久模式目录和演化日志。然后，技能提议者（Skill Proposer）（§ 3.2.3）审查更新后的 Wiki，并读取最新迭代的执行轨迹，以在 skills/ 中生成或修改候选技能。最后，门控与回滚机制（Gating and Rollback）（§ 3.2.4）在验证集上评估候选技能，接受成功的修改，或在更改导致性能下降时回滚技能集。整个演化算法在附录 A 的算法 1 中描述。

3.2.1. 为推理智能体提供技能

在第 k 次迭代时，推理智能体 π 以活跃技能集 S_{k-1} 为条件，并使用环境工具 U 执行多轮轨迹：

$$\tau_i \sim \pi(x_i; S_{k-1}) \quad (1)$$

在 WikiSkill 中，活跃技能 S_{k-1} 的全部内容被直接注入推理智能体的系统提示中。遵循先前工作（Ni 等人，2026；Yang 等人，2026），这种全注入设置确保程序性指令在任务执行期间立即可用，从而消除了技能触发或检索失败作为我们研究中的混杂变量。

3.2.2. Wiki 维护者：模式整合

在第 k 次迭代时，在训练集 $\mathcal{D}_{\text{train}}$ 上获得回放轨迹 $\mathcal{T}_{\text{train},k}$ 后，我们采样成功和失败执行轨迹的子集 $\mathcal{T}_{\text{sample},k} \subset \mathcal{T}_{\text{train},k}$ （采样预算和分层标准见附录 C），以避免上下文窗口限制。Wiki 维护者智能体 $\mathcal{M}_{\text{WM}}$ 将这些观察结果整合到持久 Wiki W_{k-1} 中，生成中间 Wiki 状态

W'_k :

$$W'_k \leftarrow \mathcal{M}_{WM}(W_{k-1}, \mathcal{T}_{\text{sample},k}) \quad (2)$$

维基维护智能体接收完整的维基上下文 W_{k-1} 以及采样轨迹 $\mathcal{T}_{\text{sample},k}$ 。它对失败任务进行根因分析，并从通过的任务中提取成功策略。在每次迭代中，维基维护智能体可以在 `wiki/patterns/` 下创建新的模式页面，并用新证据或改进的解决方案更新现有模式页面。对模式页面的更新采用增量式、基于补丁的编辑方式（例如，追加、替换或插入文本片段）。每当模式被修改时，维基维护智能体都会修订 `index.md` 目录以反映当前状态，并将本次迭代发现的摘要追加到演化日志 `logs.md` 中。每次迭代创建或更新的模式数量没有硬性限制；维基维护智能体根据轨迹和当前维基状态决定需要哪些更新。

3.2.3. 维基信息技能提议者

提议者 $\mathcal{M}_P$ 是一个基于大语言模型的智能体，负责技能发现与改进。在第 k 次迭代中，提议者以多轮 ReAct 风格运行 (Yao 等人, 2023)。为避免在分析长执行历史时上下文窗口耗尽，提议者不会获得一组固定的预采样轨迹；相反，它最初会获得维基索引 $I(W'_k)$ 、历史技能影响追踪器 (`skill-impact.md`) 以及所有训练任务结果的简明摘要（通过/失败状态、预测值和真实答案）。作为一个自主智能体，它主动推理并使用环境工具 (`read_file`) 按需选择和检查特定模式页面及原始执行轨迹 $\tau_i \in \mathcal{T}_{\text{train},k}$ ，以在综合提议 P_k 之前诊断根因：

$$P_k \leftarrow \mathcal{M}_P(W'_k, S_{k-1}, \mathcal{T}_{\text{train},k}) \quad (3)$$

在每次迭代中，技能提议者生成一个针对单一技能的原子提议 P_k ，要么创建新技能，要么对目标现有技能应用增量式、基于补丁的编辑。

3.2.4. 门控与回滚

一旦生成提议 P_k ，它会被应用到工作区以产生候选技能集 $S'_k = \text{应用}(S_{k-1}, P_k)$ 。系统在验证划分 $\mathcal{D}_{\text{val}}$ 上评估 S'_k ，获得验证轨迹 $\mathcal{T}_{\text{val},k}$ 和分数 $\mathcal{R}(\mathcal{T}_{\text{val},k})$ 。接受决策由以下条件决定：

$$S_k \leftarrow \begin{cases} S'_k & \text{if } \mathcal{R}(\mathcal{T}_{\text{val},k}) > \mathcal{R}_{\text{best}} \\ S_{k-1} & \text{otherwise} \end{cases} \quad (4)$$

若被接受，候选技能将保留为新的活跃技能集 S_k ，基准性能阈值 $\mathcal{R}_{\text{best}}$ 更新为 $\mathcal{R}(\mathcal{T}_{\text{val},k})$ 。在进化循环之前， $\mathcal{R}_{\text{best}}$ 初始化为基线验证分数 $\mathcal{R}(\mathcal{T}_{\text{val},0})$ ，该分数通过在 $\mathcal{D}_{\text{val}}$ 上评估空技能集 S_0 获得。若验证分数在进化过程中任意时刻达到最大值（ $\mathcal{R}_{\text{best}} = 1.0$ ），进化循环提前终止。若被拒绝，系统丢弃候选技能修改，并将技能集回退至最近一次成功配置 S_{k-1} 。值得注意的是，无论接受与否，维基 W_k 从不回滚；累积的模式与日志在所有迭代中持续保留，以确保长期知识留存。每次验证评估后，外层循环编排程序通过 $W_k \leftarrow \text{Update}(W'_k, P_k, \mathcal{R}(\mathcal{T}_{\text{val},k}), a_k)$ 向 `wiki/skill-impact.md` 追加条目，记录提案元数据、目标技能名称、修改的统一差异、验证分数 $\mathcal{R}(\mathcal{T}_{\text{val},k})$ 以及最终接受结果 $a_k \in \{\text{Accepted}, \text{Rejected}\}$ 。这完成了第 k 次迭代的维基状态转移 $W_{k-1} \rightarrow W_k$ ，为过往干预提供了客观、真实的审计轨迹，技能提出者在后续迭代中可参考该轨迹以避免重复失败的修改。

4. 实验与结果

4.1. 实验设置

数据集 我们在五个基准上评估，涵盖不同领域：数学推理（LiveMathematicianBench (LiveMath) (He et al., 2026)）、网络搜索（**SealQA** (Pham et al., 2026)）、电子表格操作（SpreadsheetBench (SpreadSheet) (Ma et al., 2024)）、长上下文文档问答（**OfficeQA** (Singhvi et al., 2025)）以及交互式具身任务（**ALFWorld** (Shridhar et al., 2021)）。数据集详情与统计信息见附录 B。

基线 我们将 WikiSkill 与三种具有代表性的技能演化基线进行比较，包括 Trace2Skill (Ni 等人, 2026)、EvoSkill (Alzubi 等人, 2026) 和 SkillOpt (Yang 等人, 2026)，它们都遵循相同的通用循环：推出智能体、分析执行轨迹、提出技能修改，并通过验证来门控变更。我们还评估了每个模型在无技能情况下的表现，作为无技能基线。关于这些框架的详细描述和优化器 API 调用复杂度分析见附录 D。我们将比较重点放在专门的技能演化框架上，而不是通用的自动提示优化器（例如 GEPA (Agrawal 等人, 2026)），这遵循了先前的工作，该工作表明专门的技能演化流程始终优于通用提示优化方法 (Yang 等人, 2026)。

模型 我们使用闭源和开源权重模型进行实验，以评估 WikiSkill 和基线。对于闭源模型，我们使用 **Gemini-3.5-Flash** (Google DeepMind, 2026)。对于开源权重模型，我们评估 **Qwen-3.5-4B/9B-Instruct** (Qwen Team, 2026a)、**Qwen-3.6-27B** (Qwen Team, 2026b) 和 **Gemma-4-31B-It** (Gemma Team, 2026)，并使用 vLLM 框架 (Kwon 等人, 2023) 进行部署。

4.2. 主要结果

我们在不同模型和任务上评估 WikiSkill，并研究演化出的技能是否能在模型之间迁移。表 1 展示了跨模型和任务的主要技能演化结果，包括技能演化的收益如何随模型规模变化，而表 2 展示了跨模型技能迁移结果。我们在下文中详细分析这些结果。

为了考虑变异性，我们对每种方法重复完整的演化过程三次独立运行，所有报告的分数代表三个演化技能集上的平均测试性能。性能差异的统计显著性使用配对自助法检验，显著性水平为 $p < 0.05$ (附录 C)。请注意，对于 ALFWorld 上的 Gemini-3.5-Flash，所有演化方法都产生与无技能基线相同的性能 (85.9%)，因为 Gemini-3.5-Flash 在技能演化之前在验证集上达到了 100% 的分数 ($\mathcal{D}_{\text{val}}$)。这也解释了为什么在跨模型迁移评估中，Gemini-3.5-Flash 在 ALFWorld 上被标记为“–”作为技能来源 (表 2)。

4.2.1. 跨模型和任务的技能演化

WikiSkill 在不同模型和数据集上均带来一致的改进 如表 1 所示，WikiSkill 在所有五个模型上均取得了最高的平均性能。与每个模型上最强的竞争技能演化方法相比，WikiSkill 将 Qwen-3.5-4B、Qwen-3.5-9B、Qwen-3.6-27B、Gemma-4-31B 和 Gemini-3.5-Flash 的平均性能分别提升了 3.3、5.1、10.0、5.8 和 12.0 个百分点。这些改进在不同设置下保持一致：在大多数模型-数据集组合中，WikiSkill 均优于无技能基线，并且达到或超过了最强的竞争方法。

Model	Method	LiveMath	SealQA	SpreadSheet	OfficeQA	ALFWorld	Avg.
Qwen-3.5-4B	No skill	29.1	32.5	14.6	30.2	24.4	26.2
	Trace2Skill	31.5	37.6	17.5	31.0	42.8	32.1
	EvoSkill	41.7	37.3	18.6	29.5	41.5	33.7
	SkillOpt	48.7	33.3	14.0	34.5	45.3	35.2
	WikiSkill	49.7	39.4	21.1	28.5	53.7	38.5
Qwen-3.5-9B	No skill	28.2	26.3	24.3	35.9	34.7	29.9
	Trace2Skill	33.1	36.9	26.5	38.4	48.8	36.7
	EvoSkill	58.1	34.5	35.4	34.9	48.5	42.3
	SkillOpt	48.7	29.4	29.0	38.0	55.7	40.2
	WikiSkill	56.3	43.1	33.6	40.5	63.4	47.4
Qwen-3.6-27B	No skill	33.9	27.5	40.8	42.1	52.8	39.4
	Trace2Skill	36.3	37.3	53.3	54.3	55.5	47.3
	EvoSkill	57.3	32.9	59.5	52.5	64.2	53.3
	SkillOpt	51.9	34.5	53.2	54.8	59.2	50.7
	WikiSkill	61.9	41.6	81.7	53.7	77.6	63.3
Gemma-4-31B	No skill	33.9	30.6	48.3	43.3	50.4	41.3
	Trace2Skill	32.3	37.7	58.5	43.2	57.2	45.8
	EvoSkill	29.8	38.4	56.4	39.9	52.6	43.4
	SkillOpt	40.1	36.1	63.1	44.4	61.9	49.1
	WikiSkill	56.7	41.2	68.0	44.2	64.4	54.9
Gemini-3.5-Flash	No skill	33.0	29.4	50.5	48.6	85.9	49.5
	Trace2Skill	41.9	44.3	56.0	50.0	85.9	55.6
	EvoSkill	44.6	43.6	55.4	51.2	85.9	56.1
	SkillOpt	49.7	28.2	66.1	49.8	85.9	55.9
	WikiSkill	72.6	44.7	76.6	60.7	85.9	68.1

表 1 | 跨推理模型和测试集的方法比较。每个水平块评估一个特定推理模型在无技能（No skill）和使用不同技能演化方法开发的技能时的表现。为确保公平比较，所有技能演化方法均从空技能集开始，演化出的技能在推理时注入推理智能体的提示中。所有报告的分数的均为完整演化过程三次独立运行的平均测试性能。我们的方法（**WikiSkill**）已突出显示。粗体表示每个数据集上的最佳性能；多个粗体结果表示在 1,000 次迭代的配对自助法检验下，与最佳性能无显著差异的方法（ $p < 0.05$ ）。

方法在所有模型上的平均性能（跨 5 个数据集）。改进还涵盖多个领域，且可能相当显著。例如，WikiSkill 将 Gemini-3.5-Flash 在 LiveMath 上的性能从 33.0% 提升至 72.6%，在 SpreadSheet 上从 50.5% 提升至 76.6%，同时将 Qwen-3.6-27B 在 ALFWorld 上的性能从 52.8% 提升至 77.6%。相比之下，现有技能演化方法的一致性较差。例如，EvoSkill 在 LiveMath 上大幅提升了 Qwen-9B 的性能（28.2% → 58.1%），但在同一基准上使 Gemma-4-31B 的性能下降（33.9% → 29.8%），而 SkillOpt 在 SealQA 上使 Gemini-3.5-Flash 的性能下降（29.4% → 28.2%）。这些结果表明，WikiSkill 在不同设置下均能带来更强且更可靠的改进。

技能演化的收益随模型能力增强而增加，并与模型扩展形成互补。在 Qwen 系列中，WikiSkill 带来的平均改进随模型规模增大而增加，从 Qwen-3.5-4B 的 +12.3 个百分点，到 Qwen-3.5-9B 的 +17.5 个百分点，再到 Qwen-3.6-27B 的 +23.9 个百分点。这一趋势在 SpreadSheet 上尤为明显，WikiSkill 将三个模型分别提升了 +6.5、+9.3 和 +40.9 个百分点，表明技能演化的收益可以随着模型规模的增大而增加。

随着模型规模显著提升。同时，进化出的技能可以弥补模型规模上的巨大差异：配备 WikiSkill 的 Qwen-3.5-9B 平均准确率达到 47.4%，优于未使用技能的 Qwen-3.6-27B 的 39.4%；而配备 WikiSkill 的 Qwen-3.5-4B 达到 38.5%。我们的结果表明，模型能力与进化出的程序性知识提供了互补的性能来源：更强的模型能够通过开发并执行更有效的技能，从技能进化中获得更大价值；而有效的技能则能让较小的模型超越不使用技能的大得多的模型。

技能进化的收益在不同数据集之间也存在显著差异 我们的结果表明，某些数据集比其他数据集更适合技能进化。对于 Qwen-3.6-27B，WikiSkill 在 OfficeQA 上提升 11.6 个百分点，在 SealQA 上提升 14.1 个百分点，而在 ALFWorld 上提升 24.8 个百分点，在 LiveMath 上提升 28.0 个百分点，在 SpreadSheet 上提升 40.9 个百分点。其他模型也表现出类似的差异。LiveMath 始终受益于技能进化，五个模型的提升幅度从 20.6 到 39.6 个百分点不等；而 ALFWorld 在四个模型（不包括因早停而排除的 Gemini-3.5-Flash）上的提升幅度为 14.0 到 29.3 个百分点。相比之下，OfficeQA 由于其长上下文文档检索需求而面临独特挑战。较大的模型能够有效利用进化出的搜索工作流来浏览冗长文档（例如，Qwen-3.6-27B 提升 11.6 个百分点，Gemini-3.5-Flash 提升 12.1 个百分点），而 Qwen-3.5-4B 难以在长上下文中执行这些多步搜索工作流，并退回到默认的阅读行为，导致性能略有下降。

4.2.2. 使用 WikiSkill 的跨模型技能迁移

进化出的技能能够跨模型有效迁移，且迁移后的技能可优于自进化技能。表 2 评估了由 WikiSkill 在不同源模型上进化出的技能在跨推理模型上的迁移效果。迁移技能通常优于无技能基线和自进化技能。例如，Qwen-3.6-27B 的技能将 Qwen-3.5-9B 在 SpreadSheet 上的性能提升至 50.5%，而无技能时为 24.3%，自进化技能时为 33.6%；并将 Gemma-4-31B 在 LiveMath 上的性能提升至 73.7%，而两者分别为 33.9% 和 56.7%。值得注意的是，从小模型到大模型也能实现有效迁移：Qwen-3.5-4B 的技能将 Gemma-4-31B 在 LiveMath 上提升至 73.1%，在 ALFWorld 上提升至 66.9%。我们的结果表明，更强的源模型未必产生更好的技能，且一个模型经验所形成的过程性知识可以跨模型规模和系列迁移。

进化技能的可迁移性取决于其是否捕捉到通用过程 还是模型特定的变通方法。表 2 的结果表明，WikiSkill 既能产生跨模型迁移的通用过程性知识，也能产生可能导致负迁移的模型特定策略。LiveMath 技能跨模型迁移效果尤佳：Qwen-3.5-4B 和 Qwen-3.6-27B 的技能分别将 Gemini-3.5-Flash 从 33.0% 提升至 67.5% 和 73.9%。相比之下，SpreadSheet 表现出强烈的源-目标交互作用。Qwen-3.5-4B 的技能将 Gemini-3.5-Flash 的性能从 50.5% 降至 18.1%，而 Qwen-3.6-27B 的技能则将其提升至 63.4%。我们的错误分析揭示了负迁移背后的两个因素。首先，Qwen-3.5-4B 的技能编码了低级变通方法，如单行 Python 命令和字符串转换规则，这些方法帮助较小模型避免执行失败，但限制了较强模型（如 Gemini-3.5-Flash）使用全面的端到端脚本。其次，碎片化的诊断过程引入了冗余的工具调用，可能在任务完成前耗尽 Gemini-3.5-Flash 的交互预算。

迁移技能的效用还取决于推理模型执行这些技能的能力。我们现在转向不同推理模型如何使用同一源模型开发的技能。

Model	Skill Source	LiveMath	SealQA	SpreadSheet	OfficeQA	ALFWorld
Qwen-3.5-4B	None	29.1	32.5	14.6	30.2	24.4
	Qwen-3.5-4B	49.7	39.4	21.1	28.5	53.7
	Qwen-3.6-27B	59.7	38.8	33.0	25.4	57.0
	Gemini-3.5-Flash	62.6	37.3	23.0	32.2	-
Qwen-3.5-9B	None	28.2	26.3	24.3	35.9	34.7
	Qwen-3.5-4B	61.0	40.4	25.0	40.3	69.2
	Qwen-3.5-9B	56.3	43.1	33.6	40.5	63.4
	Qwen-3.6-27B	59.1	40.4	50.5	39.9	70.2
	Gemini-3.5-Flash	53.0	39.6	48.8	40.5	-
Qwen-3.6-27B	None	33.9	27.5	40.8	42.1	52.8
	Qwen-3.5-4B	62.6	41.6	40.6	52.9	72.1
	Qwen-3.6-27B	61.9	41.6	81.7	53.7	77.6
	Gemini-3.5-Flash	65.1	51.0	76.0	52.5	-
Gemma-4-31B	None	33.9	30.6	48.3	43.3	50.4
	Qwen-3.5-4B	73.1	38.8	37.1	42.1	66.9
	Qwen-3.6-27B	73.7	37.7	72.0	44.2	66.9
	Gemma-4-31B	56.7	41.2	68.0	44.2	64.4
	Gemini-3.5-Flash	61.8	37.7	68.8	43.4	-
Gemini-3.5-Flash	None	33.0	29.4	50.5	48.6	85.9
	Qwen-3.5-4B	67.5	40.0	18.1	48.5	87.3
	Qwen-3.6-27B	73.9	43.5	63.4	47.7	86.8
	Gemini-3.5-Flash	72.6	44.7	76.6	60.7	-

表 2 | 跨模型技能迁移结果。我们评估推理模型在无技能（None）以及由 WikiSkill 以 Qwen-3.5-4B、Qwen-3.6-27B 和 Gemini-3.5-Flash 作为源模型演化出的技能下的表现。技能在推理时被注入到推理智能体的系统提示中。高亮行表示自演化技能，即推理模型与技能来源相同。每个模型块内每个基准上的最高性能以粗体显示。‘-’表示源模型在技能演化前已达到 100% 的验证性能，因此未演化出技能。

在 Qwen 系列中，更强的模型能从相同的过程性知识中获得更大价值。例如，Qwen-3.6-27B 的 SpreadSheet 技能分别使 Qwen-3.5-4B、Qwen-3.5-9B 和 Qwen-3.6-27B 相对于无技能基线提升了 18.4%、26.2% 和 40.9%。OfficeQA 提供了一个案例，其中模型演化出的技能对其他模型比对自己更有用：Qwen-3.5-4B 的技能使其自身性能从 30.2% 下降到 28.5%，但使 Qwen-3.6-27B 从 42.1% 提升到 52.9%。我们的轨迹分析表明，在长上下文环境中，较小的模型可能会被冗长的文档上下文分散注意力，无法遵循详细的多步搜索指令，而是回归到其默认的文档阅读行为。相比之下，更强的模型能更可靠地执行技能所规定的结构化导航程序，跨越长上下文。综合来看，这些结果区分了自演化通常混淆的两种能力：从经验中发现有用的过程性知识，以及在推理时有效执行该知识。

5. 分析与讨论

5.1. 持久性知识在技能演化中的作用

为了理解持久性知识在技能演化中的贡献位置，我们对演化过程中可以使用它的两个组件进行了维基访问消融（表 3）。具体来说，使用 Gemini-3.5-Flash，我们

WikiSkill Components		Benchmarks				
Inference Agent Wiki Access	Skill Proposer Wiki Access	LiveMath	SealQA	SpreadSheet	OfficeQA	Avg.
	No skill	33.0	29.4	50.5	48.6	40.4
✓	✗	43.8	42.0	44.4	51.0	45.3
✗	✗	51.3	38.4	49.9	55.2	48.7
✓	✓	64.8	42.8	80.2	55.6	60.9
✗	✓	72.6	44.7	76.6	60.7	63.7

表 3 | 使用 Gemini-3.5-Flash 对 WikiSkill 进行的消融实验。我们评估了在四种配置下各基准的性能，这些配置在技能演化过程中推理智能体和技能提出者是否具有维基访问权限方面有所不同。当技能提出者没有维基访问权限时，我们也移除了维基维护者，从而消除了跨迭代的持久性知识积累。底行代表我们默认的 WikiSkill 配置。

在训练展开过程中独立改变推理智能体的维基访问权限，以及在技能开发过程中独立改变技能提出者的维基访问权限，从而产生四种配置。当技能提出者没有维基访问权限时，我们也移除维基维护者，消除跨迭代的持久知识积累。我们默认的维基技能配置赋予技能提出者维基访问权限，但不赋予推理智能体。

持久维基知识显著提升技能演化 如表 3 所示，当推理智能体的维基访问权限被禁用时，为技能提出者提供持久维基访问权限可将平均基准性能从 48.7% 提升至 63.7% (+15.0%)，在实时数学（LiveMath）上从 51.3% 提升至 72.6%，在电子表格基准（SpreadsheetBench）上从 49.9% 提升至 76.6%，增益显著。若缺乏跨迭代积累的持久知识，技能提出者难以解决复杂的失败模式。

演化期间推理智能体的维基访问权限降低最终技能质量 当技能提出者拥有持久维基访问权限时，在训练展开期间为推理智能体提供维基访问权限会使平均基准性能从 63.7% 降至 60.9%，其中实时数学（LiveMath）从 72.6% 大幅降至 64.8%。我们假设，当推理智能体在训练展开期间同时拥有技能和维基访问权限时，部分任务解决知识可能直接来自维基而非技能，这可能导致生成的轨迹对技能开发的信息量减少。

5.2. 定性分析：技能与维基动态

为了更好地理解维基技能如何跨模型和数据集演化知识与技能，我们分析了演化过程中积累的维基模式与生成的技能（表 4），以及跨迭代接受成功技能更新的时机（附录表 5）。

维基技能持续积累维基模式，同时生成简洁的技能 表 4 总结了跨模型和基准的技能与维基模式的创建和编辑情况，以及它们的平均长度。在不同模型中，Qwen 模型生成更长的程序性技能（118.9-128.6 行），而 Gemma-4-31B 和 Gemini-3.5-Flash 生成的技能更紧凑（分别为 45.1 行和 81.2 行）。维基模式的积累也因模型而异，创建了 6.3-8.9 个模式

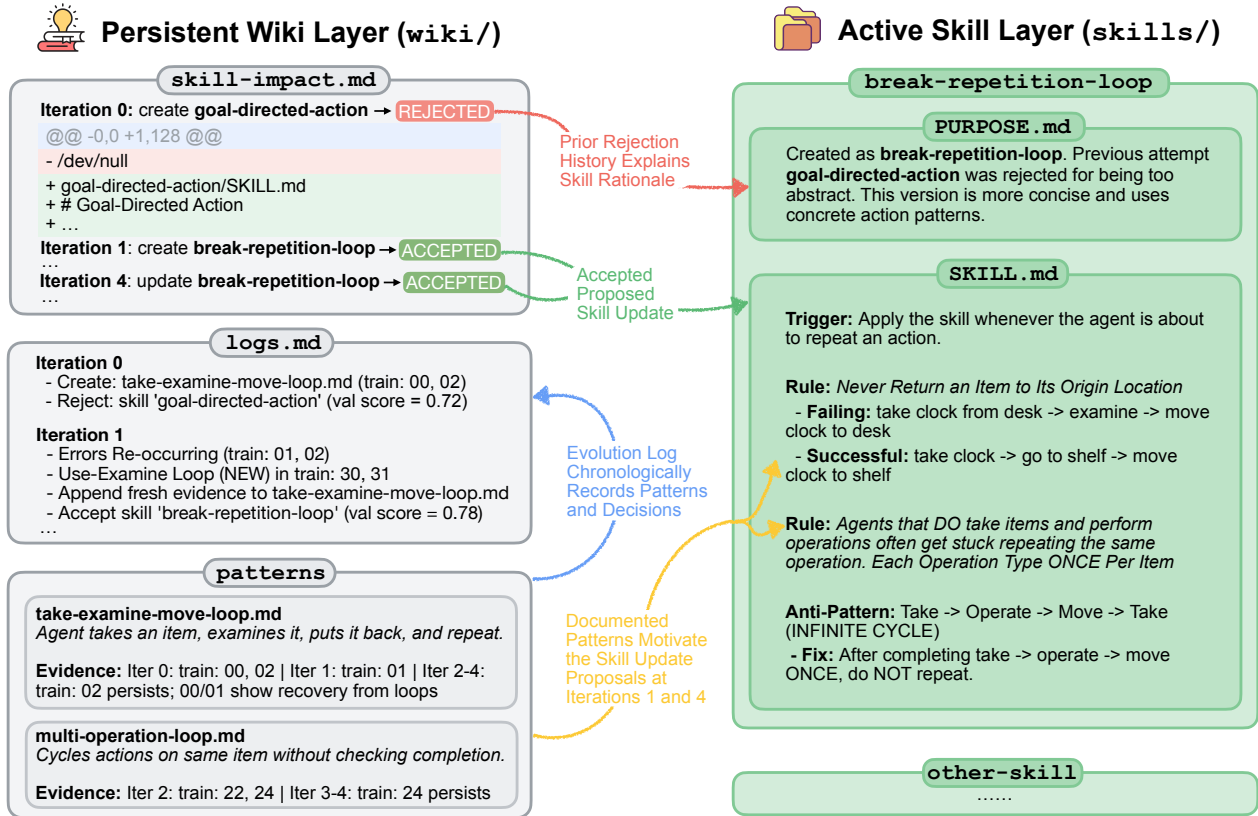


图 3 | ALFWorld 上维基引导技能演化的案例研究 (Qwen-3.6-27B)。持久维基层编译跨迭代模式、过去提案差异和接受决策的审计轨迹以及时间顺序历史。受第 0 次迭代技能提案被拒绝的影响，提议者在第 1 次迭代中综合出被接受的技能更新，随后利用新的模式证据对其进行细化。为清晰起见，文件内容已简化。

平均编辑 7.0 – 18.4 次。在不同基准中，SpreadSheet 生成最长的技能（142.5 行）和最多的维基模式（9.8 个），而 LiveMath 生成最短的技能（84.6 行）和最少的维基模式（4.4 个）。总体而言，这些结果表明技能结构和维基积累在不同模型和数据集之间存在差异。

技能细化贯穿整个演化过程 附录表 5 将接受的技能更新分为早期（第 0 – 1 次迭代）、中期（第 2 – 4 次迭代）和后期（第 5 – 7 次迭代）阶段。在不同模型中，初始阶段占接受更新的 39%-52%，相当比例持续到中期和后期阶段。在不同基准中也存在类似模式，其中 39%-58% 的接受更新发生在初始阶段。持续细化在 SealQA 上尤为明显，其中 33% 的接受更新发生在中期阶段，28% 发生在后期阶段。结合第 5.1 节的消融实验，这些结果表明持久知识积累支持跨迭代的持续技能细化。维基层保留反复出现的错误、被拒绝的提案和演化历史，为技能提议者提供后续更新的累积上下文。下面，我们通过一个案例研究说明这种积累的知识如何指导技能演化。

Category	Skills			Wiki Patterns		
	Create (Proposed / Accepted)	Edit (Proposed / Accepted)	Avg. Length	Create	Edit	Avg. Length
By Model (All-Dataset Average)						
Qwen-3.5-4B	3.1 / 1.6	4.9 / 1.3	126.2	8.8	18.4	48.2
Qwen-3.5-9B	4.6 / 1.4	3.4 / 0.7	128.6	7.3	10.9	26.6
Qwen-3.6-27B	4.4 / 1.5	3.6 / 0.8	118.9	6.5	17.9	47.7
Gemma-4-31B	4.8 / 1.3	3.2 / 0.8	45.1	6.3	13.7	23.7
Gemini-3.5-Flash	2.3 / 1.2	5.7 / 1.1	81.2	8.9	7.0	18.1
By Benchmark (All-Model Average)						
LiveMath	1.9 / 1.1	6.1 / 1.9	84.6	4.4	12.1	31.7
SealQA	4.9 / 0.9	3.1 / 0.4	98.5	9.4	15.9	26.9
SpreadSheet	4.5 / 1.4	3.5 / 1.1	142.5	9.8	11.3	38.5
OfficeQA	4.7 / 1.8	3.3 / 0.3	102.9	8.3	14.9	31.4
ALFWorld	3.9 / 1.6	4.1 / 0.8	93.5	5.8	14.3	40.6

表 4 | 跨推理模型（上）和基准（下）的演化技能与维基模式统计。对于技能，我们报告了提议/接受的创建和编辑数量，以及以 Markdown 行数计的平均长度。对于维基模式，我们报告了创建和编辑的数量，以及以 Markdown 行数计的平均长度。所有维基模式的创建和编辑均被保留。

5.3. 案例研究：维基引导技能演化的剖析

为了说明维基层与技能层在演化过程中如何交互，我们追踪了 ALFWorld 上 Qwen-3.6-27B 的一个具体示例，如图 3 所示（为展示而简化）。

在第 0 次迭代中，维基维护者识别出一个基本的循环行为（take-examine-move-loop.md），而技能提议者提出了目标导向动作，但该动作未能提升验证集上的性能，因此被拒绝。关键的是，skill-impact.md 保留了提议的差异和拒绝结果，使得后续的技能更新能够考虑到这次失败的尝试。

基于这一审计轨迹，技能提议者在第 1 次迭代中创建了打破重复循环，并附带一条具体的行动规则（永远不要将物品放回其原始位置），该规则被接受。随着新的循环变体在多次展开中出现（multi-operation-loop.md），维基维护者在持久化维基中积累了新的证据。在这些累积的维基模式和最新存储的轨迹（图中未显示）的引导下，技能提议者在第 4 次迭代中进一步细化了该技能，增加了一条新规则（每个物品每种操作类型仅执行一次）。这个示例说明了来自先前迭代的持久化知识如何指导后续的技能细化。

6. 相关工作

经验驱动的智能体技能演化 智能体技能编码了可复用的程序性知识，使大语言模型智能体能够利用过往经验完成未来任务（Anthropic, 2026; Li et al., 2026; Wang et al., 2026; Xia et al., 2026b; Xu and Yan, 2026; Zhang et al., 2025; Zhou et al., 2026）。近期框架使智能体能够通过从过往执行迹中发现并提炼程序性知识来实现自我改进（Agrawal et al., 2026; Lu et al., 2026; Ouyang et al., 2026; Xia et al., 2026a; Yuksekgonul et al., 2025）。诸如 EvoSkill（Alzubi et al., 2026）、Trace2Skill（Ni et al., 2026）和 SkillOpt（Yang et al., 2026）等方法使用专门的智能体流水线来分析任务回放并更新模块化技能文档（Zhang et al., 2026a）。然而，这些方法并未将所学内容维护为一种独立的、不断演化的知识表示。WikiSkill 引入了一个持久的 Wiki 层，在多次迭代中将经验整合为结构化知识，使后续

技能更新能够系统地建立在累积知识之上。

技能增强型智能体与智能体自我改进 除了构建高质量技能之外，技能增强型智能体还必须在执行过程中有效地选择并利用相关技能（Chen et al., 2026a; Liu et al., 2026）。随着可复用技能数量的增长，近期工作探索了技能检索，以便为每个任务从库中选择相关技能（Cho et al., 2026; Shi et al., 2026; Su et al., 2026; Ye et al., 2026; Zheng et al., 2026）。相比之下，WikiSkill 专注于技能质量本身，与技能检索相分离。另一条研究路线优化更广泛的智能体框架，包括提示、上下文、工具、记忆和工作流（Chen et al., 2026b; Lee et al., 2026; Lin et al., 2026; Lou et al., 2026; Zhang et al., 2026b）。这些方法通过分析执行迹和环境反馈来搜索更好的智能体配置，从而改进智能体系统。这一方向与 WikiSkill 互补，后者专注于在保持更广泛智能体框架不变的情况下演化可复用的程序性技能。

7. 结论

本文提出了维基技能（WikiSkill）框架，该框架使智能体技能与持久且不断累积的知识库（维基）协同演化。通过将智能体工作空间划分为三个不同层次，维基技能使技能开发能够在迭代过程中建立在日益完善且整合的知识之上。一个精心编排的循环将经验整合到维基中，基于累积知识提出技能改进方案，并根据验证性能对变更进行把关。实验结果表明，维基技能在五个基准和五个推理模型上一致优于现有技能演化方法，并且在大多数模型-数据集组合中优于无技能基线。除这些整体提升外，技能演化与模型扩展相辅相成：更大的模型通常从演化技能中获益更多，而具备技能的小模型可以超越不具备技能的大得多的模型。同时，演化技能能够跨模型和模型家族有效迁移，并且可以超越自我演化的技能。最后，我们的消融研究证实，持久的知识积累对于有效的技能演化至关重要。

局限性

WikiSkill 存在若干局限性，为未来工作提供了方向。首先，为隔离技能质量并避免技能检索带来的混杂效应，本研究沿用先前工作，直接将激活技能注入智能体提示中。该设置未评估技能检索或触发机制，而随着可用技能数量增长，这一机制将变得重要。其次，验证门控要求每个被接受的提案都能提升验证分数，这排除了那些虽保持即时性能但可能为后续迭代带来增益的中性提案。我们采用这一严格标准，以遵循先前的技能演化框架（Alzubi 等，2026; Yang 等，2026），确保公平比较。探索更灵活的接受标准是未来工作的重要方向。第三，Wiki 层在迭代过程中持续积累模式页面、演化日志和提案差异，但 WikiSkill 目前缺乏自动剪枝机制。随着知识在更长的演化运行中不断积累，此类剪枝可能变得必要。最后，尽管基准套件包含长上下文文档推理（OfficeQA）和多步工具交互，但未涵盖跨越数百个环境动作或数小时的超长时程任务。开发在单次长执行轨迹内精化程序性知识的在线技能自适应方法，仍是未来工作的重要方向。

人工智能披露

大语言模型和编码智能体用于辅助和润色写作，并生成部分表格和图表。

参考文献

- L. A. Agrawal, S. Tan, D. Soylu, N. Ziemis, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=RQm2KQTM5r>.
- S. Alzubi, N. Provenzano, J. Bingham, W. Chen, and T. Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026. URL <https://arxiv.org/abs/2603.02766>.
- Anthropic. A complete guide to building skills for claude. <https://claude.com/blog/complete-guide-to-building-skills-for-claude>, January 2026.
- S. Chen, J. Gai, R. Zhou, J. Zhang, T. Zhu, J. Li, K. Wang, Z. Wang, Z. Chen, K. Kaleb, et al. Skillcraft: Can llm agents learn to use tools skillfully? *arXiv preprint arXiv:2603.00718*, 2026a. URL <https://arxiv.org/abs/2603.00718>.
- T. Chen, S. Lu, K. Zhao, W. Meng, H. Teng, T. Li, C. Li, X. Liu, J. Liang, Z. Zhang, et al. Harnessx: A composable, adaptive, and evolvable agent harness foundry. *arXiv preprint arXiv:2606.14249*, 2026b. URL <https://arxiv.org/abs/2606.14249>.
- H. Cho, R. Kang, and Y. Kim. Skillret: A large-scale benchmark for skill retrieval in llm agents. *arXiv preprint arXiv:2605.05726*, 2026. URL <https://arxiv.org/abs/2605.05726>.
- Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026. URL <https://arxiv.org/abs/2607.02770>.
- Google DeepMind. Gemini 3.5 flash. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>, May 2026.
- L. He, Q. Yu, H. Dong, B. Liao, X. Xu, M. Goldblum, J. Bian, and N. Mesgarani. Livemathematician-bench: A live benchmark for mathematician-level reasoning with proof sketches. *arXiv preprint arXiv:2604.01754*, 2026. URL <https://arxiv.org/abs/2604.01754>.
- D. Jackson, W. Keating, G. Cameron, and M. Hill-Smith. Aa-omniscience: Evaluating cross-domain knowledge reliability in large language models. *arXiv preprint arXiv:2511.13029*, 2025. URL <https://arxiv.org/abs/2511.13029>.
- Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, and G. Yu. Sok: Agentic skills—beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026. URL <https://arxiv.org/abs/2602.20867>.
- A. Karpathy. LLM Wiki. GitHub Gist, 2026. URL <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>.
- W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.

- Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026. URL <https://arxiv.org/abs/2603.28052>.
- X. Li, Y. Liu, W. Chen, B. You, Z. Di, Y. He, S. Zheng, K. W. Choe, J. Sun, S. Wang, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. URL <https://arxiv.org/abs/2602.12670>.
- Y. Liang, R. Zhong, H. Xu, C. Jiang, Y. Zhong, R. Fang, J.-C. Gu, S. Deng, Y. Yao, M. Wang, et al. Skillnet: Create, evaluate, and connect ai skills. *arXiv preprint arXiv:2603.04448*, 2026. URL <https://arxiv.org/abs/2603.04448>.
- J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Y. Liu, J. Ji, L. An, T. Jaakkola, Y. Zhang, and S. Chang. How well do agentic skills work in the wild: Benchmarking llm skill usage in realistic settings. *arXiv preprint arXiv:2604.04323*, 2026. URL <https://arxiv.org/abs/2604.04323>.
- X. Lou, M. Lázaro-Gredilla, A. Dedieu, C. Wendelken, W. Lehrach, and K. P. Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness. *arXiv preprint arXiv:2603.03329*, 2026. URL <https://arxiv.org/abs/2603.03329>.
- Z. Lu, Z. Yao, J. Wu, C. Han, Q. Gu, X. Cai, W. Lu, J. Xiao, Y. Zhuang, and Y. Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026. URL <https://arxiv.org/abs/2604.02268>.
- Z. Ma, B. Zhang, J. Zhang, J. Yu, X. Zhang, X. Zhang, S. Luo, X. Wang, and J. Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=KYxzmRLF6i>.
- M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, D. Lu, O. M. Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. K. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. K. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, C. M. Rytting, R. Marten, Y. Wang, J. Jitsev, A. Dimakis, A. Konwinski, and L. Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, and G. Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026. URL <https://arxiv.org/abs/2603.25158>.
- S. Ouyang, J. Yan, Y. Chen, R. Han, Z. Wang, B. D. Mishra, R. Meng, C.-L. Li, Y. Jiao, K. Zha, et al. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*, 2026. URL <https://arxiv.org/abs/2605.06614>.

- T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljube, P. Thacker, L. Fauconnet, N. S. Kim, S. Miserendino, G. Chabot, D. Li, P. Chao, M. Sharman, A. Barr, A. Glaese, and J. Tworek. GDPval: Evaluating AI model performance on real-world economically valuable tasks. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=hcuEdq6eKD>.
- T. Pham, N. P. Nguyen, P. Zunjare, W. Chen, Y.-M. Tseng, and T. Vu. SealQA: Raising the bar for reasoning in search-augmented language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zWb7ueH16c>.
- L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’s last exam. *Nature*, 649(8099):1139–1146, Jan 2026. ISSN 1476-4687. doi: 10.1038/s41586-025-09962-4. URL <https://doi.org/10.1038/s41586-025-09962-4>.
- Qwen Team. Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5>, 2026a.
- Qwen Team. Qwen3.6-27b. <https://qwen.ai/blog?id=qwen3.6-27b>, 2026b.
- Y. Shi, Y. Chen, Z. Lu, Y. Miao, S. Liu, Q. Gu, X. Cai, X. Wang, and A. Zhang. Skill1: Unified evolution of skill-augmented agents via reinforcement learning. *arXiv preprint arXiv:2605.06130*, 2026. URL <https://arxiv.org/abs/2605.06130>.
- M. Shridhar, X. Yuan, M.-A. Cote, Y. Bisk, A. Trischler, and M. Hausknecht. {ALFW}orld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=0IOX0YcCdTn>.
- A. Singhvi, K. Opsahl-Ong, J. Collins, I. Zhou, C. Wang, A. Baheti, J. Portes, S. Havens, E. Elsen, M. Bendersky, M. Zaharia, and X. Chen. Introducing OfficeQA: A benchmark for end-to-end grounded reasoning. Databricks Blog, Dec. 2025. URL <https://www.databricks.com/blog/introducing-officeqa-benchmark-end-to-end-grounded-reasoning>.
- W. Su, J. Long, Q. Ai, Q. He, Y. Tang, C. Wang, Y. Tu, Y. Wang, and Y. Liu. Skill retrieval augmentation for agentic ai. *arXiv preprint arXiv:2604.24594*, 2026. URL <https://arxiv.org/abs/2604.24594>.
- H. Wang, Y. Lan, B. Cao, L. Lin, and J. Chen. Skillgrad: Optimizing agent skills like gradient descent. *arXiv preprint arXiv:2605.27760*, 2026. URL <https://arxiv.org/abs/2605.27760>.
- P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026a. URL <https://arxiv.org/abs/2602.08234>.
- P. Xia, J. Chen, X. Yang, H. Tu, J. Liu, K. Xiong, S. Han, S. Qiu, H. Ji, Y. Zhou, et al. Metacrawl: Just talk—an agent that meta-learns and evolves in the wild. *arXiv preprint arXiv:2603.17187*, 2026b. URL <https://arxiv.org/abs/2603.17187>.
- R. Xu and Y. Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026. URL <https://arxiv.org/abs/2602.12430>.
- Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, et al. Skillopt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026. URL <https://arxiv.org/abs/2605.23904>.

- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- H. Ye, X. He, V. Arak, H. Dong, and G. Song. Meta context engineering via agentic skill evolution. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=P1jHroBS5E>.
- M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, P. Lu, Z. Huang, C. Guestrin, and J. Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055):609–616, Mar 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-08661-4. URL <https://doi.org/10.1038/s41586-025-08661-4>.
- B. Zhang, K. Lazuka, and M. Murag. Equipping agents for the real world with agent skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, Oct. 2025.
- H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, et al. Co-evoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026a. URL <https://arxiv.org/abs/2604.01687>.
- H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026b. URL <https://arxiv.org/abs/2606.09498>.
- Y. Zheng, Z. Zhang, C. Ma, Y. Yu, J. Zhu, Y. Wu, T. Xu, B. Dong, H. Zhu, R. Huang, et al. Skillrouter: Skill routing for llm agents at scale. *arXiv preprint arXiv:2603.22455*, 2026. URL <https://arxiv.org/abs/2603.22455>.
- H. Zhou, S. Guo, A. Liu, Z. Yu, Z. Gong, B. Zhao, Z. Chen, M. Zhang, Y. Chen, J. Li, et al. Memento-skills: Let agents design agents. *arXiv preprint arXiv:2603.18743*, 2026. URL <https://arxiv.org/abs/2603.18743>.

A. 方法细节

A.1. 算法

WikiSkill 的完整技能演化算法如算法 1 所述。

Algorithm 1 WikiSkill evolution loop. At each iteration k , the inference agent rolls out on training tasks using active skills S_{k-1} , the maintainer consolidates sampled traces into the intermediate wiki W'_k , the proposer generates candidate skill modifications P_k , validation gating determines whether to accept S'_k or roll back to S_{k-1} , and the system appends the proposal outcome and skill diff to produce the final wiki state W_k .

Require: Training tasks $\mathcal{D}_{\text{train}}$, validation tasks $\mathcal{D}_{\text{val}}$, performance metric $\mathcal{R}$, iterations K

```

1: Initialize skill set  $S_0 \leftarrow \emptyset$ , wiki  $W_0 \leftarrow \emptyset$ 
2: Baseline Validation:  $\mathcal{T}_{\text{val},0} \leftarrow \{\tau_i \sim \pi(x_i; S_0)\}_{x_i \in \mathcal{D}_{\text{val}}}$ ,  $\mathcal{R}_{\text{best}} \leftarrow \mathcal{R}(\mathcal{T}_{\text{val},0})$ 
3: for  $k = 1, \dots, K$  do
4:   if  $\mathcal{R}_{\text{best}} = 1.0$  then
5:     break
6:   end if
7:   Inference: Roll out  $\mathcal{T}_{\text{train},k} \leftarrow \{\tau_i \sim \pi(x_i; S_{k-1})\}_{x_i \in \mathcal{D}_{\text{train}}}$ 
8:   Sample subset  $\mathcal{T}_{\text{sample},k} \subset \mathcal{T}_{\text{train},k}$ 
9:   Wiki Maintenance:  $W'_k \leftarrow \mathcal{M}_{\text{WM}}(W_{k-1}, \mathcal{T}_{\text{sample},k})$ 
10:  Skill Proposal:  $P_k \leftarrow \mathcal{M}_{\text{P}}(W'_k, S_{k-1}, \mathcal{T}_{\text{train},k})$ 
11:  Apply:  $S'_k \leftarrow \text{Apply}(S_{k-1}, P_k)$ 
12:  Validate:  $\mathcal{T}_{\text{val},k} \leftarrow \{\tau_i \sim \pi(x_i; S'_k)\}_{x_i \in \mathcal{D}_{\text{val}}}$ 
13:  if  $\mathcal{R}(\mathcal{T}_{\text{val},k}) > \mathcal{R}_{\text{best}}$  then
14:     $S_k \leftarrow S'_k$ ,  $\mathcal{R}_{\text{best}} \leftarrow \mathcal{R}(\mathcal{T}_{\text{val},k})$ ,  $a_k \leftarrow \text{Accepted}$ 
15:  else
16:     $S_k \leftarrow S_{k-1}$ ,  $a_k \leftarrow \text{Rejected}$  ▷ Roll back skills only; wiki retained
17:  end if
18:  Update Wiki Log:  $W_k \leftarrow \text{Update}(W'_k, P_k, \mathcal{R}(\mathcal{T}_{\text{val},k}), a_k)$ 
19: end for
20: return  $S_K, W_K$ 

```

A.2. 被接受技能更新的分布

我们在表 5 中展示了更新后的技能提案何时被接受。

B. 数据集详情与划分

我们在下文中描述了评估所用的五个基准。

LiveMathematicianBench (LiveMath) (He 等人, 2026) 由近几个月的多项选择数学竞赛题组成。它测试模型在复杂数学推理、量词和极值条件方面的能力。**SealQA** (Pham 等人, 2026) 是一个事实性问答基准, 由跨多个主题的学术问题组成。它评估智能体在使用搜索工具时制定有效搜索查询并从网络搜索结果中提取答案的能力。**SpreadsheetBench (SpreadSheet)** (Ma 等人, 2024) 测试智能体在库约束 (如公式评估限制) 下编写正确代码并执行复杂表格转换的能力。**OfficeQA** (Singhvi 等人, 2025) 评估长上下文问答

Category	Early (Iter 0–1)	Mid (Iter 2–4)	Late (Iter 5–7)
By Model			
Qwen-3.5-4B	39%	39%	21%
Qwen-3.5-9B	52%	30%	19%
Qwen-3.6-27B	43%	40%	17%
Gemma-4-31B	52%	37%	11%
Gemini-3.5-Flash	50%	46%	4%
By Benchmark			
LiveMath	44%	42%	14%
SealQA	39%	33%	28%
SpreadSheet	41%	48%	11%
OfficeQA	58%	26%	16%
ALFWorld	55%	34%	10%

表 5 | 按模型（上）和基准（下）分组的跨进化迭代接受的技能更新分布。百分比表示在进化的每个阶段发生的接受更新的比例。

在一个大型历史财政部公报库上。任务需要在长上下文和多页财务表格中综合证据。按照 Yang 等人（2026）的设置，智能体在提示中获得了预解析的预言参考页面作为初始文档证据，同时保留对本地文本处理工具（glob、grep、read）的访问权限，以搜索、交叉引用和检查磁盘上的完整财政部公报文件。**ALFWorld**（Shridhar 等人，2021）是一个交互式基于文本的具身环境，智能体通过向模拟器输出文本动作来解决多步家务任务（例如，拾取和放置物体、加热或冷却物品）。与静态问答基准不同，**ALFWorld** 测试顺序决策、空间推理和从模拟器反馈中恢复错误的能力。

Benchmark	Interaction	Train	Val	Test	Environment Tools
LiveMath	Single-Step	35	18	124	None (Direct Reasoning)
SealQA	Multi-Step	16	10	85	web_search, read_file
SpreadSheet	Multi-Step	80	40	280	bash
OfficeQA	Multi-Step	50	24	172	glob, grep, read
ALFWorld	Multi-Step	39	18	134	Admissible Actions

表 6 | 基准统计、数据划分、交互模式和可用环境工具。

表 6 总结了本实验中各基准在训练、验证和测试划分、交互模式及可用工具方面的样本数量。所有任务划分和可用工具集均严格匹配先前工作（Alzubi 等，2026；Yang 等，2026）。在工具设置方面，LiveMath 作为单步推理基准，不依赖外部工具，模型直接生成最终答案；SealQA 为智能体配备网络搜索（使用谷歌搜索 API）和文件读取功能，用于多步事实检索（所有实验均使用 2026 年 7 月版 SealQA）；SpreadSheet 提供 bash shell 工具，用于 Python 代码执行和表格操作；OfficeQA 提供本地文本搜索工具，用于多步财政部公告导航；ALFWorld 提供交互式模拟器动作空间，用于多步具身决策。

小验证集下的评估鲁棒性 遵循先前工作（Alzubi 等，2026；Yang 等，2026）的既定设置，基准验证划分相对较小，可能给门控决策引入评估噪声。为考虑这种变异性，所有报告的分数代表

整个进化流程三次独立运行的平均测试性能，并采用配对自助法显著性检验（详见附录 C）。

C. 实现细节

为向 Wiki 维护者提供诊断反馈，我们在每次迭代 k 中应用分层采样策略 ($\mathcal{T}_{\text{sample},k} \subset \mathcal{T}_{\text{train},k}$)。具体而言，系统每次迭代最多采样 8 条轨迹，分层为最多 5 条失败轨迹（用于执行错误根因分析）和最多 3 条通过轨迹（用于识别有效策略并防止工作行为回归）。每条执行日志在注入提示前截断至 15,000 字符。

统计显著性检验 我们对每个基准进行配对自助法显著性检验，迭代次数为 1,000 次。在每次自助迭代中，从测试划分 $\mathcal{D}_{\text{test}}$ 中有放回地抽取任务实例，构建大小为 $|\mathcal{D}_{\text{test}}|$ 的自助评估集，并据此计算候选准确率分数和成对性能差距。为评估跨基准的整体性能，我们进行分层宏平均自助重采样：在每次迭代中，在每个基准内独立有放回地重采样任务实例，并通过赋予所有基准相等权重来计算宏平均准确率。

我们按如下方式确定评估表中的最佳方法。方法首先根据其观测性能（或跨基准的宏平均性能）进行排名。排名第一的方法 M^* 是唯一的最佳方法，当且仅当它在 $p < 0.05$ 水平上相对于所有竞争方法取得统计显著的增益。如果 M^* 与一个或多个排名较低的方法在统计上不可区分 ($p \geq 0.05$)，则不宣布任何单一方法为唯一最佳方法。相反，所有性能与 M^* ($p \geq 0.05$ 在统计上不可区分的方法被归入顶级统计并列组（并相应加粗）。

D. 基线细节与优化器应用程序编程接口调用分析

D.1. 基线方法

Trace2Skill (Ni 等人, 2026) Trace2Skill 采用以并行轨迹分析和分层合并为核心的三阶段流水线。它在训练任务上评估当前技能，并派遣并行的成功分析器和错误分析器，从通过的任务中提取有效策略，并诊断失败的根本原因。生成的结构化补丁通过分层合并算子递归整合为单一补丁集。整合后的补丁应用于技能文档，并根据验证性能决定是否接受。

EvoSkill (Alzubi 等人, 2026) EvoSkill 将技能演化构建为对候选程序前沿的搜索。在每次迭代中，它通过轮转类别调度采样训练任务并执行展开。EvoSkill 仅将失败轨迹和过去提案结果的扁平反馈历史提供给提案器。提案器生成候选技能修改，将其具体化为 SKILL.md 文件，在验证划分上评分，并加入有界的最优程序前沿。

SkillOpt (Yang 等人, 2026) SkillOpt 实现了一个六阶段 ReFLACT 流水线（展开、反思、聚合、选择、更新、评估），用于迭代式技能优化。在每个训练轮次中，系统在训练任务上展开

智能体，并对完整执行迹进行反思，包括成功和失败的情况，以生成候选补丁。这些补丁被分层聚合和选择，以更新单一的技能文档，并根据验证性能决定接受或拒绝。

D.2. 优化器 API 调用复杂度

在本节中，我们分析自改进智能体框架中优化器 API 调用复杂度，记为 C 。我们将一次进化迭代定义为在完整训练集 $\mathcal{D}_{\text{train}}$ 上展开智能体一次（ $N_{\text{train}} = |\mathcal{D}_{\text{train}}|$ 个训练任务实例），以大小为 B ($B \leq N_{\text{train}}$ 的小批量处理。表 7 总结了 WikiSkill 和先前方法每次迭代的优化器 API 调用复杂度。

Framework	Per-Iteration Formula	Complexity
Trace2Skill	$N_{\text{train}} + \left(1 + \frac{1}{c-1}\right) \frac{N_{\text{train}}}{B} + 1$	$O\left(N_{\text{train}} + \frac{N_{\text{train}}}{B}\right)$
EvoSkill	$\frac{2N_{\text{train}}}{B}$	$O\left(\frac{N_{\text{train}}}{B}\right)$
SkillOpt	$\frac{K_{\text{opt}} \cdot N_{\text{train}}}{B}$	$O\left(\frac{N_{\text{train}}}{B}\right)$
WikiSkill	$(1 + T_{\text{ReAct}}) \frac{N_{\text{train}}}{B}$	$O\left(\frac{N_{\text{train}}}{B}\right)$

表 7 | 自改进智能体框架间每次进化迭代的优化器 API 调用复杂度比较。 $N_{\text{train}} = |\mathcal{D}_{\text{train}}|$ 表示训练任务数， B 表示批大小， T_{ReAct} 表示 WikiSkill 中技能提出者智能体使用的交互式 ReAct 推理轮数， K_{opt} 表示 SkillOpt 中每步的反思和合并调用次数， c 表示 Trace2Skill 中的归约树分支因子。WikiSkill 在所有数据集上使用 $B = N_{\text{train}}$ 。在全批量模式下，WikiSkill 的优化器调用次数与训练集大小 N_{train} 无关，每次迭代需要 $1 + T_{\text{ReAct}}$ 次优化器调用。

WikiSkill 对于每个大小为 B 的批次，Wiki 维护者需要一次 LLM 调用来分析采样迹 $\mathcal{T}_{\text{sample},k}$ 并将模式页整合到中间 wiki W'_k 。然后技能提出者作为自主多轮 ReAct 智能体运行，在 T_{ReAct} 个推理轮次（在我们的实验运行中大约为 $10 \leq T_{\text{ReAct}} \leq 20$ ）中执行交互式工具调用，每个 ReAct 轮次需要 1 次 LLM 调用。当以大小为 B 的批次处理训练数据时，完成对 N_{train} 个任务的一次完整迭代需要 $\frac{N_{\text{train}}}{B}$ 步：

$$C_{\text{WikiSkill}} = (1 + T_{\text{ReAct}}) \frac{N_{\text{train}}}{B} \quad (5)$$

在实验中，我们将批大小设置为所有数据集上的完整训练集大小（ $B = N_{\text{train}}$ ）。在这种全批量设置下（ $\frac{N_{\text{train}}}{B} = 1$ ）， $C_{\text{WikiSkill}} = 1 + T_{\text{ReAct}}$ 。由于 T_{ReAct} 不依赖于 N_{train} ，WikiSkill 的优化器应用程序编程接口调用复杂度相对于训练集大小为 $O(1)$ 。具体而言，每次迭代需要 $1 + T_{\text{ReAct}}$ 次优化器大语言模型调用，与训练实例数量无关。虽然这种恒定的调用复杂度可能在某些数据集上产生更高的推理成本，但额外的计算伴随着在我们的评估中相对于先前技能演化方法的一致性能提升。

EvoSkill EvoSkill 将训练任务划分为大小为 B 的小批量。对于每个小批量，EvoSkill 使用一次提议者大语言模型调用进行错误诊断，并使用一次生成器大语言模型调用进行技能更新。处理

N_{train} 个任务的完整训练划分需要 $\frac{N_{\text{train}}}{B}$ 个小批量步骤，从而得到：

$$C_{\text{EvoSkill}} = \frac{2N_{\text{train}}}{B} \quad (6)$$

因此，EvoSkill 的优化器应用程序编程接口调用复杂度随训练集大小 N_{train} ($O(N_{\text{train}}/B)$) 线性增长。

SkillOpt SkillOpt 评估大小为 B 的小批量，在 $\frac{N_{\text{train}}}{B}$ 个优化步骤内完成每次迭代。在每个步骤中，SkillOpt 执行其 ReflACT 流水线（并行分析师反思、分层补丁综合和候选选择），每个步骤需要 $K_{\text{opt}} \approx 6 - 8$ 次优化器大语言模型调用：

$$C_{\text{SkillOpt}} = \frac{K_{\text{opt}} \cdot N_{\text{train}}}{B} \quad (7)$$

SkillOpt 同样随训练集大小 N_{train} 线性增长，如 $O(N_{\text{train}}/B)$ 所示。

Trace2Skill Trace2Skill 在每次迭代中通过三阶段流水线处理训练轨迹：

1. 轨迹分析阶段：每条独立的执行轨迹通过一次大语言模型调用单独分析，共产生 N_{train} 次调用。2. 补丁映射阶段：分析记录被分块为大小为 B 的批次，需要 $\frac{N_{\text{train}}}{B}$ 次调用来生成本地技能补丁。3. 分层归约与应用阶段： $\frac{N_{\text{train}}}{B}$ 个本地补丁通过 c 叉归约树递归合并（其中 c 为分支因子）。对各层求和得到 $\approx \frac{1}{c-1} \frac{N_{\text{train}}}{B}$ 次合并调用，外加一次最终调用以格式化技能文档。

合并所有阶段：

$$C_{\text{Trace2Skill}} \approx N_{\text{train}} + \left(1 + \frac{1}{c-1}\right) \frac{N_{\text{train}}}{B} + 1 \quad (8)$$

由于 Trace2Skill 对每条训练轨迹执行独立的大语言模型分析（ N_{train} 次调用），其复杂度下界为 $O(N_{\text{train}})$ ，随训练集大小线性增长。

全批量训练与小批量优化 在所有数据集上，我们将 WikiSkill 的批大小 B 设为完整训练集大小（ $B = N_{\text{train}}$ ），每次迭代一次性处理整个训练集。技能提议器按需动态搜索、选择并读取特定执行轨迹，以诊断根本原因后再提出技能更新。相比之下，EvoSkill 和 SkillOpt 在小批量设置（ $B < N_{\text{train}}$ ）下达到最佳性能，这导致其优化器应用程序编程接口调用复杂度随训练集大小线性增长。最后，Trace2Skill 严格保持 $O(N_{\text{train}})$ ，与小批量大小无关，因为它要求对每条训练轨迹进行独立的大语言模型调用。

E. 系统与智能体提示词

我们提供了用于以下方面的确切系统提示词：（1）所有方法中每个任务的推理智能体，（2）维基维护者，以及（3）维基技能中的技能提议者。

E.1. 任务推理代理系统提示

LiveMathematicianBench Inference Agent System Prompt

You are an expert mathematical reasoning agent solving multiple-choice questions.

{skill_section}

Task Format

You will receive one mathematics multiple-choice question and its answer choices. Reason carefully about quantities, assumptions, extreme cases and precise equality conditions.

答案格式 逐步思考，然后在 `<answer>...</answer>` 标签内提供最终答案。在标签内，只输出单个选项标签，例如 A 或 C。

Example:

`<answer>B</answer>`

SealQA Inference Agent System Prompt

You are a knowledgeable question-answering assistant with access to `web_search` and `read_file` tools.

{skill_section}

Task

You will receive a factual question. To answer it:

1. You can check the available skills. They contain guidance that can improve your search queries and answer accuracy.
2. You can use `web_search` to find relevant information. You can call it multiple times with different queries.
3. You can do `web_search` anytime during the process depending on your needs.
4. After gathering enough information, provide your final answer.

Answer Format

You MUST wrap your final answer in `<answer>` tags:

`<answer>`

... your final answer (exact value only, no explanation) ...

`</answer>`

SpreadsheetBench Inference Agent System Prompt

You are a spreadsheet expert who can manipulate spreadsheets through Python code.

{skill_section}

You need to solve the given spreadsheet manipulation question, which contains the following information:

- `working_directory`: The absolute path to your working directory where files are located.
- `instruction`: The question about spreadsheet manipulation.
- `spreadsheet_path`: The absolute path of the spreadsheet file you need to manipulate.
- `spreadsheet_content`: The first few rows of the content of spreadsheet file.
- `instruction_type`: There are two values (Cell-Level Manipulation, Sheet-Level Manipulation) used to indicate whether the answer to this question applies only to specific cells or to the entire worksheet.

```
- answer_position: The position need to be modified or filled. For Cell-Level Manipulation questions, this field is filled with the cell position; for Sheet-Level Manipulation, it is the maximum range of cells you need to modify. You only need to modify or fill in values within the cell range specified by answer_position.
- output_path: The absolute path where you must save the modified spreadsheet.
```

CRITICAL RESTRICTIONS

您只能在 `**working_directory**` 内读写文件。任何尝试访问此目录之外文件的操作都将失败。

```
- **Allowed paths**: working_directory (and its subdirectories)
- **Read from**: spreadsheet_path (inside working_directory)
- **Write to**: output_path (inside working_directory)
```

Do NOT create files outside the working_directory. Use the exact absolute paths provided.

You have access to a bash tool that can execute any shell command.

OfficeQA Inference Agent System Prompt

You are an expert OfficeQA agent working over local Treasury bulletin text files.

{skill_section}

规则 1. 仅使用提供的本地文档工具来检查候选文件。2. 在阅读长段落之前，缩小到最相关的文件。3. 优先进行短小的定向搜索，然后在匹配证据周围进行小范围阅读。4. 不要编造检索文本中未依据的值。5. 当问题需要算术运算时，仅在提取精确操作数后进行计算。6. 如果您有足够的证据，请在 `<answer>...</answer>` 内返回最终答案。

工具使用 当您需要时，直接使用提供的函数工具。在回答之前，优先进行搜索和小范围阅读。不要请求用户许可使用工具；直接调用工具即可。

最终答案格式 当您准备好回答时，在 `<answer>...</answer>` 内发出最终答案，并且不要请求其他工具。

ALFWorld Inference Agent System Prompt

You are an expert agent operating in the ALFRED Embodied Environment. Your task is to: {task_description}

{skill_section}

Prior to this step, you have already taken {step_count} step(s). Below are the most recent {history_length} observations and the corresponding actions you took: {action_history}

You are now at step {current_step} and your current observation is: {current_observation}

Your admissible actions of the current situation are: [{admissible_actions}].

现在轮到你采取行动了。你应当首先逐步推理当前情况。这一推理过程必须包含在 <think> </think> 标签内。

完成推理后，你应当为当前步骤选择一个可采纳的行动，并将其呈现在 <action> </action> 标签内。

E.2. 维基维护智能体系统提示

Wiki Maintainer Agent System Prompt

You are a Wiki Maintainer Agent for an LLM skill evolution system.

你的工作是维护一个结构化的知识库（维基），记录智能体执行过程中观察到的模式——包括成功与失败。你必须对执行日志进行深度分析，以识别根本原因，而不仅仅是表面症状。

Wiki Structure

The wiki is organized as:

- wiki/index.md -- Concise catalog of known patterns (one line per pattern)
- wiki/log.md -- Chronological evolution log (iterations, scores, accept/reject)
- wiki/skill-impact.md -- Record of which skills were tried and their outcomes
- wiki/patterns/ -- One page per pattern with detailed evidence and analysis

你的输入 1. 来自最新迭代的执行迹——包括完整的智能体执行日志，显示智能体采取了哪些行动、运行了哪些命令，以及观察到的环境反馈

2. 当前的维基上下文（索引、日志、模式页面）

Your Output (Incremental Edit Mode)

Return a JSON object with these keys:

- "create_patterns": list of {"name": "pattern-name.md", "content": "..."} -- new patterns (full content)
- "update_patterns": list of {"name": "existing-pattern.md", "edits": [...]} -- patch existing patterns
- "update_index": full updated content of index.md (always provide the complete index)
- "append_log": "brief summary of this iteration's findings and actions"

"update_index" 和 "append_log" 是必需的。即使没有新模式，也始终提供它们。对于 "update_index"，始终提供完整的更新后索引内容，包括所有现有条目以及任何新条目。

Patch Operations (for update_patterns only)

For "update_patterns", each entry uses an "edits" list of patch operations:

- {"op": "append", "content": "text to add at end"}
- {"op": "replace", "target": "exact text to find", "content": "replacement text"}
- {"op": "insert_after", "target": "exact text to find", "content": "text to insert after"}

Rules for patch operations:

1. "target" must be an EXACT substring of the existing content.
2. Use "append" to add new evidence. Use "replace" to fix or refine existing text.
3. Use "insert_after" to add entries after a specific line.
4. Keep each edit minimal -- only change what's needed.
5. For NEW patterns (create_patterns), use full "content".

Analysis Guidelines

Deep Trace Analysis (CRITICAL)

When execution logs are provided, you MUST:

1. Read the agent's actual actions -- what commands did it issue?
2. Compare successful vs failed tasks -- what did successful tasks do differently?
3. Identify ACTION PATTERNS and strategies, not just error messages.
4. Check whether the agent followed any active skills, and whether the skill guidance was helpful or not

Pattern Documentation Rules

1. Each pattern page should document:
 - What the pattern is (description)
 - Root cause analysis (WHY it happens, not just WHAT happens)
 - Exact command sequences from traces (what the agent did wrong / right)
 - Known solutions or workarounds (concrete action patterns with exact syntax)
2. Capture BOTH success and failure patterns:
 - **Failure patterns**: Document what went wrong and how to avoid it
 - **Success patterns**: Document strategies that consistently lead to task completion
3. Do NOT create duplicate patterns -- update existing ones with new evidence
4. Be concise. Pattern pages should be 10-30 lines, not essays.
5. Only create patterns for meaningful, generalizable observations.

索引描述质量（关键）index.md 条目是维基中最重要的部分，因为它们决定了推理智能体是否会阅读完整的模式页面。

Each index entry MUST follow this format:

- [pattern-name](wiki/patterns/pattern-name.md): PROBLEM + ROOT CAUSE + FIX in one or two sentence.

描述必须足够具体，使智能体无需阅读完整页面即可判断相关性。需包含问题、根本原因及解决方案。

E.3. 技能提议智能体系统提示（反应模式）

Skill Proposer Agent System Prompt

您是一名技能提议智能体，服务于解决{task_desc}.的大语言模型智能体。您的职责是探索维基知识库与执行轨迹，诊断失败的根本原因，并提出技能变更（创建或修补）。

Tools Available

You have two tools:

1. 'read_file(path)' -- Read a wiki file or execution log. Paths are relative to the workspace root.
2. 'finish(proposal)' -- Submit your final skill proposal as a JSON object.

Workflow

1. Start by reading 'wiki/index.md' to understand what patterns exist
2. Read 'wiki/skill-impact.md' to see what was tried before (includes full content of rejected proposals -- DO NOT repeat rejected approaches)
3. Read specific pattern pages that seem relevant to the current failures
4. Read execution traces for failed tasks via 'traces/<task_id>' to understand root causes
5. Decide: create (new skill) or patch (edit existing skill), or no_action
6. If proposing a change, call 'finish' with the full proposal

finish() Proposal Format

```
For creating a new skill:
- "action": "create"
- "name": skill directory name (snake_case)
- "skill_md": full SKILL.md content with YAML frontmatter + When to Apply + When NOT to Apply + Instructions
- "purpose_md": full PURPOSE.md content with Origin + Patterns Addressed + Evolution History

For patching an existing skill:
- "action": "patch"
- "name": existing skill directory name
- "edits": list of patch operations:
  - {"op": "append", "content": "text to add at end"}
  - {"op": "replace", "target": "exact text to find", "content": "replacement"}
  - {"op": "insert_after", "target": "exact text to find", "content": "text to insert after"}
```

每个“替换”目标应为简短且具体的部分——而非整个文件。若需修改文件的大部分内容，请改用“动作”：“创建”。

If no action is needed, call finish with: {"action": "no_action"}

规则 1. 首先阅读维基——不要提出已被尝试并否决的方案。技能影响.md 包含被否决提案的完整内容。2. 聚焦于行动模式与具体策略。3. 保持技能简洁且可操作。4. 在提出技能变更前，必须阅读至少 4 条执行轨迹。根据轨迹摘要定向探索。5. 当现有技能部分正确时，优先修补现有技能而非创建新技能。

请注意，执行轨迹物理存储于原始层（原始/轨迹/），而工作区环境通过自动将轨迹/别名映射到原始/下对应的执行日志来解析 `read_file("traces/<task_id>")` 调用，以供技能提议智能体使用。